Le Resume Studio

Developer Guide

October 12 2026 | Published app version 17

This guide documents the deployed static app, its HTML and CSS structure, JavaScript modules, data contracts, browser APIs, export pipeline, tests, and publishing process. It describes the implementation as it exists, including its limits, so a developer can maintain it without assuming a backend or AI service.

1 Runtime and local setup

Technology	Actual role
HTML5	index.html and download.html define semantic forms, previews, native dialogs, navigation, and save controls.
CSS	styles.css, accessibility.css, and browser.css provide the visual system, reflow, focus, contrast, and mobile adjustments.
JavaScript	Native ES modules run the entire application in the visitor’s browser. There is no transpilation or app build step.
Vendored libraries	docx.js writes DOCX files; pdf-lib.min.js writes PDFs. Nimbus Sans font assets support the PDF layout. JSZip 3.10.1 reads DOCX; PDF.js 5.6.205 extracts PDF text. Reader assets and licenses are served locally.
Node.js	Development tooling runs node:test checks and export validation scripts. It is not an application server.
Python and Java	Neither is used by the app runtime. Python builds these guides with python-docx and can run a local static server or document QA. No Java backend or native Android project is present.

The repository package is an ES-module package with a test script and no installed application dependency requirement. Use a recent Node.js release for development. Serve dist over HTTP or HTTPS; opening index.html as a file URL is not a supported module-loading workflow.

```
npm test
# Optional local server from the repository root
python3 -m http.server 8000 --directory dist
# Open http://localhost:8000
```

Production serves dist through Sites using .openai/hosting.json. There is no application REST API, database, account synchronization, OpenAI API, or required API key. Network requests load static files and export assets. The app’s API consists of the JavaScript interfaces described below and browser APIs listed in Section 5.

© 2026 Marabou Lane LLC

2 Source map and presentation layers

File or folder	Responsibility
dist/index.html	Main page, input/output sections, six review tabs, eight dialogs, labels, and export controls.
dist/app.js	DOM orchestration, upload-only master, saved history, document edits, reset lifecycle, clipboard, and export dialogs.
dist/resume.js	createBlankMaster and roleHeaderFields. Uploaded header order is retained; source headings retain role/company/years.
dist/privacy.js	Web Crypto password-derived encrypted vault, unlock, write queue and lock invalidation.
dist/imports.js	imports.js reads DOCX/PDF/TXT/RTF and validates facts; formatting.js detects bounded typography and section order.
dist/engine.js	Keyword catalog and detection, evidence support, prioritization, tailoring, cover letters, text output, and scoring.
dist/writing.js	The HR writing profile, nine action-verb categories, conservative English edits, and writing review.
dist/ats.js	46 named systems plus general profile, shared template rules, source-linked guidance, and file checks.
dist/exports.js	Resume and cover-letter DOCX/PDF generation, font embedding, text wrapping, and pagination.
dist/downloads.js	Payload validation, filenames, Blob creation, private session transfer, save links, and optional file sharing.
dist/download.html and download-page.js	Standalone download page receiving an exact document snapshot.
dist/accessibility.js	Dialog focus, announcements, field errors, and focus restoration.
dist/browser.js	Clone fallback, lazy export-library loading, and visible viewport tracking.
dist/vendor	Browser export libraries, font assets, and relevant font license files.
tests	node:test suites, DOM adapter, parsing helpers, source fixtures, and export QA scripts. Not deployed.
README and review Markdown files	Implementation notes for ATS, accessibility, and browser compatibility.

HTML and CSS maintenance

Both pages declare English, allow viewport zoom, and load styles.css, accessibility.css and browser.css in that order. They use JavaScript modules. Preserve stylesheet order for layout, focus, contrast, reflow and mobile overrides.

© 2026 Marabou Lane LLC

3 Data contracts and stored records

Master and resume objects are plain serializable data. The blank factory returns empty strings and arrays. A readable upload saves only after required source facts validate. Imports add source, importedText, importedFileName and formatting metadata. Clear Master Resume writes an explicit blank marker, clears source text and current output, and prevents history recovery from restoring the deleted master. Saved versions retain separate snapshots until cleared.

```
const master = {
  name: "Avery Taylor", phone: "",
  email: "avery@example.com", linkedin: "",
  source: "Uploaded master resume",
  summary: "Business analyst supporting reporting.",
  competencies: ["SQL", "Requirements Analysis"],
  roles: [{
    employer: "Example Company", title: "Business Analyst",
    dates: "2022 to Present",
    bullets: ["Analyzed SQL reports for stakeholders."]
  }],
  skills: [{group: "Analysis", items: ["SQL"]}],
  education: [{school: "Example University",
    qualification: "Bachelor of Science", dates: "2020"}]
};
```

The example is fictitious. A source upload requires name, summary, a competency and complete employer/title/dates/bullets for every role. Recognized education entries require all three fields. Contact details and skill groups are optional; empty skill groups are omitted. UI limits are 30 roles and 20 skill groups or education entries. These UI limits are not a substitute for validating stored or transferred data.

Record	Important fields
Application	id, createdAt, resume, master, jd, title, company, length, extra, ats, keywords, changes, englishCorrections, writingProfile, coverLetter, audit, coverAudit, review.
Cover letter	date, company, subject, salutation, paragraphs as strings, closing, signature; edited marks customized content.
Keyword	Detected jobTerms, normalized concept name, evidence/support status, and inclusion information. Preserve the evidence flag when rescoring.
Audit	total, before, after, supported, coverage, score, targetMet, targetPossible, possibleCoverage, phrases, termsNeeded, evidenceNeeded.

Store records as JSON. cloneData uses structuredClone when available and a JSON clone fallback for these plain objects. Startup shape checks discard corrupt history entries and cap history at ten. Do not treat externally edited localStorage as trusted executable content.

© 2026 Marabou Lane LLC

Master resume import interfaces

```
validateResumeFile(file) -> extension or throws
extractResumeText(file, {signal, loadZip, loadPDF, onFormatting} = {})
  -> Promise<string>
parseResumeText(text) -> parsed source object
```

File validation accepts PDF, DOCX, TXT and RTF with a 10 MB limit. TXT is strict UTF-8. RTF decodes Windows-1252 and supported Unicode controls, excludes metadata and embedded objects, and never executes content. DOCX uses JSZip with a 30 MB expanded archive limit. PDF.js reads up to 30 pages with evaluation disabled and local assets. No OCR is provided. Text is capped at 200,000 characters.

Parsing recognizes common sections and confirmed three-field employment headings. Word numbering preserves bullet markers; PDF text is ordered by row and horizontal position. Until a complete next heading is confirmed, text remains with the current role, including its final accomplishment. headerOrder preserves role, employer and date order. A complete upload saves automatically; missing required facts produce an error without replacing the prior saved source.

app.js guards reads with an epoch and AbortController. Clear, dialog close and workspace reset invalidate pending reads; late completion cannot restore deleted content. Upload disables the picker while reading; Clear can cancel a pending read. The input clears before its chooser opens and after completion, allowing repeated selection of the same file. Filename and applicant name are never duplicate blockers. Progress, associated errors and a read-only text view support keyboard access. Clear resets the file selection, extracted text, master marker and current output; stored versions remain separate.

PDF.js and its worker must be served as modules with standard_fonts and licenses. Readers load only when needed. If DOCX or PDF reading fails, revise the file or upload TXT or RTF; manual master entry is unavailable. formatting.js normalizes bounded typography, margins, page size, labels and section order. Word uses detected font settings; PDF infers a compatible serif, sans or mono family. Tables, graphics and arbitrary original positions are not reproduced.

TXT and RTF use a single column, Arial 11 point and 0.75 inch margins, within MIT recommended font and margin ranges. This fallback does not guarantee ATS acceptance.

MIT Resume Checklist

4 Tailoring engine API and score behavior

Primary JavaScript interfaces

```
tailorResume(master, jd, {
  title = "", company = "", length = "balanced",
  extra = "", ats = "general"
} = {}) -> Application
```

Synchronous entry point. Rejects fewer than 80 trimmed characters or 12 words, more than 40,000 characters, or a master lacking a name or role. balanced retains up to four first-role bullets and three later-role bullets; detailed keeps all. It clones the master, creates both documents, and returns reviews. Catch its Error and show a readable message without enabling exports.

```
extractKeywords(jd, master, extra = "") -> Keyword[]
auditResume(resume, keywords, master) -> Audit
auditCoverLetter(letter, keywords) -> Audit
reviewApplication(result, {reextract = false} = {})
  -> result with refreshed audits and writing review
```

Detection combines a finite catalog with named-tool and stated-skill detection. Additional comma-separated terms help recover missed phrases. Catalog concepts can have multiple employer aliases; every unique normalized job phrase is scored. Additional terms absent from the posting are excluded from the denominator. Resume evidence controls support; typing an unsupported term into the output cannot create evidence.

```
createCoverLetter(application) -> CoverLetter
resumeText(resume) -> string
coverLetterText(letter, resume) -> string
```

The letter draws on supported accomplishments and job terms. Text serializers provide the source for TXT exports and clipboard content. Cover scoring examines paragraphs rather than the subject or contact metadata. reviewApplication mutates the review fields on the passed application; it does not replace a customized letter.

Pipeline and invariants

The engine detects phrases, checks evidence, focuses the summary, moves relevant competencies first, ranks bullets within each role, rewrites equivalent supported wording, orders skills, inserts missing supported phrases, polishes English, creates the letter, and audits both documents. Original employers, titles, dates, metrics, and qualifications are preserved. Support work must retain its responsibility level.

The 10/10 target requires a nonzero denominator and every detected phrase to be both supported and included. Percent coverage and the 0-to-10 score are floored to one decimal, so an incomplete match cannot round up to 10. Unsupported phrases stay in the denominator; zero detected phrases does not count as success. The catalog is finite, and the overlap measure is not a live ATS score or semantic evaluation of every job requirement.

writing.js exposes polishResume, polishCoverLetter, cleanEnglish, polishBullet, actionVerb, firstPersonBullet, and writingReview. The HR profile models a hiring reviewer with over twenty years of experience as an editorial lens. It neither adds that tenure to the applicant nor performs human review. Grammar rules are conservative; unrecognized openings and long sentences remain review items.

© 2026 Marabou Lane LLC

5 Export interfaces and browser APIs

```
createDownloadPayload(application, kind, format) -> Payload
validateDownloadPayload(payload) -> payload or throws
createExportBlob(payload, dependencies = {}) -> Promise<Blob>
encodeDownloadPayload(payload) -> base64url string
decodeDownloadPayload(hash) -> validated Payload
downloadPageUrl(payload, host) -> private URL string
configurePrivateHandoff(link, payload, status, host) -> cleanup
readDownloadTransfer(host = globalThis) -> Payload
```

kind is resume or cover; format is docx, pdf, or txt. The version-1 payload contains name, resume, letter, and ats. Resume payloads contain the selected resume; cover payloads contain contact data and the letter. They exclude the full master, job posting, and saved history. Filenames combine applicant, job title, company, and document type, sanitize punctuation, and add the extension.

Validation constrains supported formats, filename characters and extension, contact strings, arrays, and letter paragraphs. configurePrivateHandoff opens a same-origin tab on the user's click. A 600,000-byte maximum snapshot is stored in sessionStorage under a random UUID for five minutes; only #transfer=UUID enters the URL. The opened tab inherits a copy of the source sessionStorage, consumes the transfer, clears its URL and retains the snapshot for refresh. The parent removes its transfer record. A copied URL in another session fails. Blocked popups or sessionStorage must offer direct save. Legacy encode/decode helpers remain internal serialization utilities and are not used in public links.

```
createDocx(resume, docxApi) -> Promise<Blob>
createCoverDocx(letter, resume, docxApi) -> Promise<Blob>
createPdf(resume, pdfApi, fontPrograms) -> Promise<Uint8Array>
createCoverPdf(letter, resume, pdfApi, fontPrograms)
  -> Promise<Uint8Array>
```

createExportBlob wraps PDF bytes in the proper Blob type and loads the requested local library. TXT uses no export library. Source formatting travels with resume payloads and snapshots; cover letters keep their professional template. PDFs embed matching CFF font programs: Nimbus Sans or PDF.js serif/mono assets. Non-ASCII normalization may alter characters, so the review recommends Word or TXT when needed.

Browser API	Purpose and fallback
localStorage and sessionStorage	Browser-owned master/history and temporary download snapshot. Catch blocked access or quota errors.
Blob and URL object URLs	Prepare downloadable bytes. Revoke old links when replaced, cleared, or reset.
showSaveFilePicker	Optional save picker outside frames; confirms completion after stream.close. Otherwise use a download link [1].
navigator.share and canShare	Optional exact-file sharing on a direct click; hidden if unsupported or embedded [2].
navigator.clipboard.writeText	Automatic copy when permitted; otherwise show selectable text [3].
HTMLDialogElement and VisualViewport	Modal focus and mobile keyboard sizing. CSS remains the viewport fallback [4,5].

© 2026 Marabou Lane LLC

6 HTML CSS and editor implementation

Preserve semantic relationships

index.html uses labeled forms, a skip link, navigation and main landmarks, alert regions for validation, and status regions for asynchronous updates. The review tablist contains Resume, Cover letter, Keywords, Writing review, ATS review, and Changes. Each tab controls a named panel. app.js keeps one tab selected, uses a roving tabindex, and handles Left/Right/Home/End. Preserve these relationships when adding or renaming UI elements [6].

```
<label for="job-description">Please add the job description
  so your resume can be tailored.</label>
<textarea id="job-description" required minlength="80"
  maxlength="40000"
  aria-describedby="jd-helper jd-requirements char-count">
</textarea>
```

Custom JavaScript also validates the word count because the form uses novalidate. Editors identify the invalid field, associate its error, expand a closed role if necessary, and focus it. Named dialog headings receive initial focus; closing restores focus to the opener. Status messages are announced inside an active modal instead of its inert background [7].

Upload and clear controls

The master dialog contains a labeled native file picker, read-only extracted text and Clear Master Resume. There is no master form, manual Save or Add/Remove handler. buildEditor and collectEditor are used only for tailored output. Source corrections happen in the original file and are uploaded again. Clear invalidates pending reads before awaiting storage and resetWorkspace empties previews, extracted text, picker and prepared links.

Responsive CSS and focus

The base layout uses a sidebar and a two-column workspace. Overrides use minmax(0, ...), min-width:0, wrapping controls, relative type sizes, and narrow-screen stacking. Browser rules switch layouts around 1000 and 640 CSS pixels, add safe-area insets, and account for short landscape screens. Keep form text at least 16 CSS pixels at default size and preserve browser zoom.

```
/* Representative rule from browser.css */
@media (max-width:640px) {
  .export-actions {
    display:grid;
    grid-template-columns:repeat(2,minmax(0,1fr));
  }
}
```

Accessibility rules enlarge targets, show clear focus indicators, support forced colors and reduced motion, and make scrollable previews keyboard reachable. Viewport tracking updates --visible-height and --visible-top when the mobile keyboard changes the visible area; pinch zoom removes overrides. These measures require real-device and assistive-technology verification before a conformance claim.

© 2026 Marabou Lane LLC

7 State lifecycle privacy and failure handling

Storage key	Contents and lifetime
lw-resume-studio-master-v1	Saved master or an explicit Blank master resume marker. localStorage for this origin and browser profile.
lw-resume-studio-history-v1	Up to ten complete application records. Each includes a master snapshot and both documents.
lw-resume-studio-export-v1	Selected document snapshot in the standalone download tab’s sessionStorage.
lw-resume-studio-draft-v1	Retired. Removed at startup; job drafts are no longer persisted.

Startup and resets

When protection is off, load a valid saved master or recover this browser’s history snapshot. An explicit blank marker prevents recovery after clearing. A protected workspace starts blank until unlocked. Master readiness controls tailoring; new browser profiles receive no applicant data.

resetWorkspace runs at startup and on pageshow, including back/forward-cache restoration. It clears job fields, output, audits, dialogs, manual-copy text, preferences, and prepared download links, while retaining master/history. workspaceEpoch guards asynchronous tailoring and export completion so a reset or deletion cannot repopulate cleared output. Preserve these guards when adding new asynchronous work.

Deletion and storage errors

Clear saved versions deletes the history key when unprotected, or awaits an encrypted empty-history write when protected. It clears memory, output, editors and prepared links, cancels pending work and reports persistence failures. Clear Master Resume stores a blank source and keeps history. Other-tab storage changes invalidate stale copies.

If storage is blocked or full, reads and writes are caught, a persistent notice appears, and the current tab can continue tailoring and downloading. Saving or clearing a master reports whether persistence failed. The user must download copies before the tab closes. Existing downloaded files and separate download-tab snapshots are outside history deletion.

Security and sharing boundaries

Applicant data stays local. Opt-in password protection encrypts saved records and removes legacy plaintext. Without it, users sharing a browser profile can read saved data. Normal app and download URLs contain no applicant content. Site sharing exposes app code; no applicant database or cloud account exists.

Escape stored text, validate transfers and avoid executable interpolation. Both HTML pages use no-referrer metadata and same-origin script/connection policies, block object embeds and external form submission, and load local vendors. No service worker or native offline package is shipped.

© 2026 Marabou Lane LLC

Password protected storage API

```
PrivateVault(storage, crypto)
  exists() -> boolean
  create(passphrase, {master, history}) -> Promise<void>
  unlock(passphrase) -> Promise<{master, history}>
  write({master, history}) -> Promise<void>
  flush() -> Promise<void>
  lock() -> void
```

privacy.js uses Web Crypto AES-GCM with a 256-bit nonextractable key, a random 16-byte salt and a fresh 12-byte IV for each write. PBKDF2-HMAC-SHA-256 derives the key with 600,000 iterations. The minimum passphrase length is 12 characters. localStorage key le-resume-studio-private-vault-v1 contains schema, salt, IV, work factor and authenticated ciphertext; it contains no plaintext master, history, password or key. Authentication failure rejects wrong passwords and modified ciphertext.

Writes serialize through a promise queue and capture a JSON snapshot. Lock increments an epoch and drops the key; a pending encryption cannot write after locking. Encrypted writes replace the vault atomically only after encryption succeeds. Upload and deletion controls await persistence. Other-tab vault updates invalidate an unlocked copy. Storage failure preserves the current tab and exposes an error rather than claiming durable protection. Protection setup and unlock both retry deletion of legacy plaintext keys.

This is local passphrase protection, not a server login or a password recovery system. It does not hide an unlocked tab, protect downloads on the filesystem, secure a compromised device, or prevent a trusted future app script from reading an unlocked workspace. Browser session restoration and actual popup/iframe behavior require real-device verification. Close download tabs when handing over a shared device. Keep the app origin and vendored assets trusted.

OWASP password storage guidance

MDN sessionStorage and opener inheritance

8 Verification deployment and maintenance

Checks and release procedure

The upload and source formatting release passed 126 node:test checks on October 12 2026. Tests cover tailoring evidence, both document scores, English rules, actual export bytes, transfers, save failures, history deletion, reset guards, keyboard semantics, capability fallbacks, and seven fresh-master journeys. DOM adapters and capability simulations do not substitute for browser rendering or real ATS uploads.

1. Run npm test. If export code changes, also run the applicable export-check scripts and inspect actual Word/PDF output, extracted text, page breaks, and fonts.

2. Confirm dist contains only deployable assets. Keep tests and personal fixtures out of the hosting archive. Review README, ATS_COMPATIBILITY, ACCESSIBILITY, and BROWSER_COMPATIBILITY when behavior changes.

3. Use the Sites source workflow to obtain a temporary repository write credential, open the checkout, commit and push changes, and package the static dist directory.

4. Save the Site version with its project ID, exact commit SHA, and archive, then deploy that saved version. Preserve the existing sharing audience. Inspect deployment status and keep the previous saved version available for rollback.

The Sites management API is a publishing interface, separate from the browser app: sites_create_source_repository_write_credential opens authorized source access; sites_save_site_version records commit_sha and archive; sites_deploy_site_version publishes the version_id. Keep credentials out of source files, URLs, logs, and client JavaScript. The hosting manifest uses static.directory = dist.

Safe extension points

- Add job vocabulary in engine.js with explicit evidence and phrase-boundary tests. Test alternative terms and unsupported qualifications; do not lower the score denominator to meet a target.
- Extend writing.js conservatively. Test tense, English punctuation, metrics, and responsibility level against the source.
- Update ATS profiles only with current primary vendor guidance. Most selected systems use general guidance; this app has no live ATS integration or certification.
- For a future AI service, synchronized accounts, or native app, design a separate authenticated backend, data handling, and mobile packaging plan. None of these components exists in the present app.

Primary API and accessibility references

[1] MDN showSaveFilePicker

[2] MDN Navigator share

[3] MDN Clipboard writeText

[4] MDN HTML dialog

[5] MDN VisualViewport

[6] W3C tabs pattern

[7] W3C modal dialog pattern

© 2026 Marabou Lane LLC